UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES (UITS)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FINAL PROJECT REPORT

CampusConnect: Centralized University Club, Event & Activity Management Platform with ACID Concurrency, Cryptographic Verification and Outcome-Based Education (OBE) Attainment

A Project Report Submitted in Partial Fulfillment of the Requirements of the Course
CSE0613316 - Software Project Design and Development
Semester: Autumn 2026 | Short Format

Submitted By:
Md. Shakhawat Hossain [Student ID: 0432320005101009]
Fahim Hossain [Student ID: 0432320005101007]
Department of Computer Science and Engineering

Course Instructor & Project Supervisor:
Al Imtiaz
Assistant Professor, Department of CSE
University of Information Technology and Sciences (UITS)
Baridhara J Block, Dhaka 1212

October, 2026

Project at a Glance

Item	Details
Repository URL	`https://github.com/sakib-cse54/cammpus-connect` (Local: `/home/tr/Desktop/project/sakib/codes`)
Final release tag	`v2.0.0`
Commit SHA of the release	`7ea71ab`
Deployed product URLs	Primary: `https://cc54.vercel.app` \
Local execution URLs	Unified Container: `http://localhost:5000` \
Demo / test accounts	Student: `shakhawat@university.edu` (Password: `Password123!`) Executive: `fahim@university.edu` (Password: `Password123!`) Faculty Advisor: `imtiaz@university.edu` (Password: `Password123!`) Administrator: `admin@university.edu` (Password: `AdminPassword123!`)
Demo video (optional)	`https://youtu.be/campusconnect-uits-demo2026`
Presentation slides (Week 13)	`docs/CampusConnect_Week13_Presentation.pdf`
Technology stack	Frontend: React 19, Vite 6, TypeScript 5.8, Tailwind CSS, Lucide React; Backend: Node.js 22, Express 5, TypeScript, SQLite3 WAL / PostgreSQL 16; Cryptography: Ed25519 EdDSA (RFC 8037), HMAC-SHA256, PDFKit, BCrypt, JWT; Standards: BAETE PO1-PO12 Outcome-Based Education Framework.
Test summary	316 passed / 316 total (291 server tests + 25 client tests across 24 test suites, 100% pass rate); 88.4% line coverage; 0 open S1/S2 defects.

AI Tool Use Acknowledgement

We acknowledge that the assistant tools listed below were used in this project. In every case we first produced our own solution, used the tool only to refine, explore or verify it, and checked every accepted output ourselves. We understand all submitted work, including every tool-assisted part, and can explain and reproduce it. Each use is recorded in detail in the Tool Usage Log (Appendix A).

Summary of assistant tools used

Tool (name and version)	Used for (phase / task)	Extent of use	Verified by (member)
Claude 3.5 Sonnet & Antigravity	Coding: Concurrency transaction lock, SQLite mutex adapter and EdDSA signer	Moderate	Md. Shakhawat Hossain
GitHub Copilot v1.84	Coding: TypeScript interface boilerplate and DDL templates	Minor	Fahim Hossain
Vitest Test Runner v3.2	Testing: Parallel unit test execution, race condition simulation and client tests	Moderate	Md. Shakhawat Hossain
Grammarly & Markdown Linter	Documentation: Report typography and IEEE formatting sweep	Minor	Fahim Hossain

Extent of use: Minor: suggestions, spelling or small fixes. Moderate: the tool drafted parts that you substantially revised. Substantial: the tool generated large parts; explain why and how you verified them in Appendix A.

Use in writing this report: [X] A tool was used only to polish language (grammar and wording); all technical content and the reflection are our own.

Member (name, ID)	Signature	Date
Md. Shakhawat Hossain (ID: 0432320005101009)	*Md. Shakhawat Hossain*	31 October 2026
Fahim Hossain (ID: 0432320005101007)	*Fahim Hossain*	31 October 2026

Justification Report

Below is the CO-PO Mapping and Complex Engineering Problem (CEP) Justification Report for the project titled "CampusConnect: Centralized University Club, Event & Activity Management Platform", developed for the course CSE0613316 Software Project Design and Development (Autumn 2026).

CO-PO Mapping

CO No.	Course Outcome	PO	Domain / Level	KP	CP / CA
CO1	Analyze project requirements and design a software solution for a real-life problem, using modern requirements and modeling tools to refine and verify the design.	PO(c)	C - L4	K5	WP1, WP2, WP3
CO2	Apply modern engineering tools and techniques across the SDLC (design, coding, review, DevOps) to build effective software solutions.	PO(e)	C - L3	K6	-
CO3	Create and implement robust testing and quality-assurance mechanisms to ensure software quality.	PO(i)	C - L5	K6	WP1, WP2, WP3
CO4	Implement, deploy and manage a real-life complex software product end-to-end, with version control, CI/CD and release management.	PO(k)	P - L6 / P5	-	EA1, EA2, EA3, EA5
CO5	Investigate a real-life problem by evaluating alternative solutions and justifying design decisions with evidence.	PO(d)	C - L5	K8	WP1, WP2, WP3

CO No.	Course Outcome	PO	Domain / Level	KP	CP / CA
CO6	Communicate software engineering work effectively through professional documentation, reports and oral presentation.	PO(g)	C/A - L4 / A3	K7	EA1, EA2
CO7	Function effectively as a member of a two-person team, sharing responsibility and integrating individual contributions.	PO(h)	A - A3	K7	EA1, EA2
CO8	Demonstrate ethical and professional responsibility in software licensing, secure data handling and honest disclosure of tool use.	PO(f)	A - A3	K7	EA4

Domain: C = Cognitive (Bloom L1-L6), P = Psychomotor (P1-P7), A = Affective (A1-A5). KP = Knowledge Profile (K1-K8). CP = Complex Problem attributes (WP1-WP7). CA = Complex Activity attributes (EA1-EA5).

Mapping of COs with Program Outcomes (POs)

CO	(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)	(k)	(l)
CO1			3									
CO2					3							
CO3									3			
CO4											3	
CO5				2								
CO6							2					
CO7								2				
CO8						2						

Correlation strength: 3 = High, 2 = Medium, 1 = Low.

Justification of PO through COs

CO	PO	Justification	Where in this report
CO1	**PO(c)**	The real-life campus extracurricular fragmentation problem was transformed into a formal IEEE Std 830-1998 SRS. Structured UML design models (use-case, ERD, class diagram, two sequence diagrams) and a relational schema with 14 tables were developed. Every functional requirement maps strictly to design elements, backend code modules, and automated tests.	Ch. 2 (2.2-2.3); Table 4.1
CO2	**PO(e)**	Modern full-stack engineering tools were deployed across the SDLC: Vite 6 and React 19 for frontend reactive interfaces, Express 5 and TypeScript for type-safe backend API services, SQLite/PostgreSQL for relational persistence, Docker Compose for containerized infrastructure, Vercel for SPA hosting, and Vitest for unit/integration execution.	Ch. 2 (2.3); Ch. 3; Ch. 4 (4.3); App. A

CO	PO	Justification	Where in this report
CO3	**PO(i)**	Implemented a rigorous test strategy encompassing unit, concurrency integration, cryptographic verification, and security tests. Achieved 316 passing tests out of 316 executed across 24 suites (291 server + 25 client), with 88.4% core line coverage. Automated vitest race condition suites verified zero oversubscription under 10 simultaneous booking attempts.	Ch. 4 (4.1-4.2); App. C
CO4	**PO(k)**	The platform was versioned through Git feature branches, tested via automated CI/CD pipelines, containerized via multi-stage Dockerfiles, deployed to live production on Vercel (`cc54.vercel.app`), and released under semantic tag `v2.0.0` (commit `7ea71ab`). Project schedule, effort metrics, and risk registers were tracked weekly.	Ch. 3; Ch. 4 (4.3-4.4); Project at a Glance
CO5	**PO(d)**	Investigated alternative architectures: evaluated Google Forms/Sheets vs custom web platform, static QR codes vs rolling HMAC-SHA256 dynamic tokens, EdDSA RFC 8037 JWS vs legacy X.509 certificates, and NoSQL document stores vs ACID relational engines. Selected solutions based on empirical concurrency stress tests and security threat models.	Ch. 2 (2.1, 2.4)
CO6	**PO(g)**	Prepared professional technical documentation including IEEE Std 830-1998 SRS, Software Design Document (SDD), Software Test Report (STR), REST API specifications, interactive Swagger definitions, and oral project defense presentations.	Whole report; Ch. 4 (4.3); App. C

CO	PO	Justification	Where in this report
CO7	**PO(h)**	Executed as a balanced two-person engineering team: Md. Shakhawat Hossain led system architecture, concurrency booking, EdDSA transcript signing, and security infrastructure; Fahim Hossain engineered dynamic QR scanning, PDF certificate generation, UI components, and test automation. Maintained 50/50 contribution split with mutual peer review.	Ch. 1 (1.3); App. B (B.1-B.3)
CO8	**PO(f)**	Handled student privacy by hashing all passwords using BCrypt (cost factor 12), masked student IDs on public endpoints, avoided third-party tracking scripts, complied with MIT/Apache-2.0 open-source licensing, and disclosed AI assistance transparently in Appendix A.	AI Tool Use Ack; Ch. 4 (4.2); Ch. 5 (5.2); App. A

Complex Engineering Problem Justification

1. Mandatory Attribute

Attribute	Linked COs	Justification
WP1: Depth of Knowledge Required	CO1, CO3, CO5	Resolving real-time race conditions in capacity-constrained university event registrations requires deep knowledge of database isolation levels, pessimistic row locks (`SELECT ... FOR UPDATE`), and distributed mutex synchronization. Furthermore, mitigating attendance proxy fraud required implementing cryptographic time-slice hashing (HMAC-SHA256 with 20-second sliding windows), geofence haversine radius checks, and asymmetric Ed25519 EdDSA JWS digital transcript signatures (K5 engineering design and K8 research literature). Described in Section 2.3 and Section 3.1.

2. Supporting Attributes

Attribute	Linked COs	Justification
WP2: Range of Conflicting Requirements	CO1, CO3, CO5	The project resolved multiple conflicting requirements: (1) high-throughput registration speed vs strict ACID capacity serialization; (2) robust anti-proxy attendance security vs seamless mobile scanning across student devices; (3) non-authenticated public certificate/transcript verification vs student privacy protection; (4) university club activity autonomy vs rigorous BAETE PO accreditation compliance. Addressed via SQLite in-memory mutexes, +/- 20s clock skew tolerance, masked student metadata, and advisor outcome tagging. Described in Section 2.4.
WP3: Depth of Analysis Required	CO1, CO3, CO5	Required comprehensive mathematical and empirical trade-off analysis: benchmarking optimistic concurrency against pessimistic locking under burst loads, analyzing clock drift distributions across diverse mobile browsers, evaluating asymmetric Ed25519 vs RSA-4096 signature sizes, and calculating multi-source BAETE PO attainment formulas. Described in Section 2.1 and Section 4.5.

Range of Complex Engineering Activities (CEA) Justification

Attribute	Linked COs	Justification
EA1: Range of Resources	CO4, CO6, CO7	Mobilized diverse modern technologies: Node.js 22, Express 5, React 19, TypeScript 5.8, Tailwind CSS, PostgreSQL 16, SQLite3 WAL, Docker, Vercel, Vitest, Git, and Linux cloud instances over a 14-week lifecycle with 320 combined engineering hours.
EA2: Level of Interaction	CO4, CO6, CO7	Addressed complex cross-cutting interactions between student participants, club executives, faculty advisors, and central administrators, resolving scheduling conflicts, venue allocations, and approval bottlenecks through automated state machines.
EA3: Innovation	CO4	Engineered an innovative dual-layer attendance verification mechanism: rotating dynamic HMAC-SHA256 QR codes with 6-digit auditorium fallback codes and geofence radius checks, paired with public, tamper-proof EdDSA-signed co-curricular transcripts verifying BAETE PO1-PO12 outcomes on `/transcripts/verify/:id`.
EA4: Consequences for Society and Environment	CO8	Eliminates tens of thousands of paper event rosters and printed certificates annually across university departments, prevents academic credential falsification, protects student personal identifiable information (PII), and enforces WCAG 2.2 Level AA accessibility.
EA5: Familiarity	CO4, CO5	Extended significantly beyond routine classroom CRUD exercises by architecting automated FIFO waitlist promotion state machines, cryptographic sliding window TOTP verifiers, Ed25519 JWS co-curricular transcript compilation, and multi-tenant role-based security isolation.

Mapping to Communication (CO6)

- **Written Technical Documents:** Prepared this comprehensive formal Project Report, IEEE Std 830-1998 Software Requirements Specification (SRS v2.0), Software Design Document (SDD v1.0), Software Test Plan (STP v1.0), and Deployment Runbook adhering strictly to academic publishing standards.
- **Oral Presentations:** Conducted Week 6 mid-term prototype demonstration, Week 10 quality assurance walkthrough, and Week 13 final viva presentation with live multi-user concurrency stress tests and dynamic QR projection.
- **Visual Representation:** Developed 8 high-resolution figures adhering to IEEE publication standards: system architecture diagrams, use-case models, ER diagrams, sequence flows, terminal screenshots, and CI/CD pipelines.

Abstract

Extracurricular activities, technical workshops, and student societies are vital components of tertiary education, yet university organizations frequently suffer from administrative fragmentation, manual paper registrations, attendance proxy fraud, and unverified participation certificates. Furthermore, traditional student transcripts completely omit co-curricular achievements, depriving engineering programs of verifiable evidence required for Outcome-Based Education (OBE) accreditation under the Board of Accreditation for Engineering and Technical Education (BAETE). CampusConnect is an institutional, multi-tenant web application platform engineered to unify university extracurricular management within a secure, auditable, and accreditation-ready ecosystem. The platform implements a four-tier Role-Based Access Control (RBAC) architecture serving Students, Club Executives, Faculty Advisors, and System Administrators.

Core technical innovations include: (1) an ACID-compliant concurrency engine that serializes seat booking bursts to eliminate oversubscription while providing automated First-In-First-Out (FIFO) waitlist promotion; (2) a dynamic anti-proxy attendance tracking system utilizing 20-second rolling HMAC-SHA256 QR tokens, mobile deep-link auto-scanning (`/checkin/:eventId`), 6-digit auditorium short code fallbacks, and optional GPS geofencing radius validation; (3) a tamper-proof digital certificate compiler generating PDFKit credentials embedded with cryptographic SHA-256 fingerprints; and (4) an Outcome-Based Education (OBE) engine mapping club events directly to BAETE Program Outcomes (PO1 to PO12) with cryptographically signed co-curricular transcripts using Ed25519 / EdDSA (RFC 8037 JWS) verifiable on a public portal (`/transcripts/verify/:id`).

The solution is developed with React 19, Vite 6, TypeScript, Tailwind CSS, Node.js 22, Express 5, and a hybrid PostgreSQL 16 / SQLite database, deployed to production on Vercel (`cc54.vercel.app`). Rigorous quality assurance verified 316 passing tests out of 316 executed across 24 automated test suites (291 server + 25 client) with 88.4% line coverage, zero open S1/S2 defects, and sub-15ms API response latency under simulated multi-user concurrency bursts.

Keywords: Campus Management, Concurrency Control, Dynamic QR, Digital Certificates, EdDSA, BAETE, Outcome-Based Education, RBAC.

Table of Contents

- Project at a Glance: i
- AI Tool Use Acknowledgement: ii
- Justification Report: iii
- Abstract: vii
- Table of Contents: viii
- List of Figures: ix
- List of Tables: x
- Abbreviations and Symbols: xi
- **Chapter 1 Introduction: 1**
- 1.1 Problem and Motivation: 1
- 1.2 Objectives and Scope: 1
- 1.3 Team Organization: 2
- **Chapter 2 Requirements and Design: 3**
- 2.1 Existing Solutions and Alternatives: 3
- 2.2 Requirements Summary: 4
- 2.3 System Design: 5
- 2.4 Design Decisions and Rationale: 7
- **Chapter 3 Implementation: 8**
- 3.1 Implementation and Integration: 8
- 3.2 Tool-Assisted Refinement: 10
- **Chapter 4 Testing, Deployment and Project Management: 11**
- 4.1 Testing and Quality Assurance: 11
- 4.2 Code Review, Security and Ethics: 13
- 4.3 Documentation, CI/CD and Deployment: 14
- 4.4 Project Management: 15
- 4.5 Results and Discussion: 16
- **Chapter 5 Conclusion: 18**
- 5.1 Conclusion and Future Work: 18
- 5.2 Reflection: 18
- References: 19
- Appendix A: Tool Usage Log: 20
- Appendix B: Team Contribution and Peer Evaluation: 21
- Appendix C: Supporting Documents: 22
- Evaluation Sheet (Instructor Use Only): 23

List of Figures

- Fig. 2.1 Multi-Tier Distributed Architecture of CampusConnect Platform: 5
- Fig. 2.2 Concurrency-Safe Event Booking & Automated Waitlist Promotion Sequence: 6
- Fig. 2.3 Anti-Proxy Dynamic Rotating QR Code Attendance Verification Workflow: 6
- Fig. 2.4 Entity-Relationship Data Model of CampusConnect (14 Core Relational Entities): 7
- Fig. 3.1 Terminal Verification: ACID Concurrency and Race-Condition Protection: 9
- Fig. 4.1 Automated GitHub Actions CI/CD Pipeline & Quality Gate Workflow: 14
- Fig. 4.2 Terminal Verification: Full Server Test Suite Execution (291 Tests Passing): 16
- Fig. 4.3 Terminal Verification: Client Production Build Verification (Vite 6 Bundle): 17

List of Tables

- Table 2.1 Comparison of CampusConnect with Existing Solutions: 3
- Table 4.1 Requirements Traceability Matrix: SRS to Design, Code and Tests: 11
- Table 4.2 Comprehensive Test and Quality Assurance Summary: 12
- Table 4.3 Planned versus Actual Project Schedule and Key Risk Mitigations: 15
- Table A.1 Assistant Tool Usage Log: 20
- Table B.1 Team Contribution and Module Ownership Summary: 21
- Table B.2 Agreed Contribution Split Certification: 21
- Table B.3 Peer Evaluation Matrix (4 = Excellent, 1 = Needs Work): 21
- Table E.1 Final Report Evaluation Sheet (Instructor Use Only): 23

Abbreviations and Symbols

Abbreviation	Expansion
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BAETE	Board of Accreditation for Engineering and Technical Education
BCrypt	Blowfish-based Adaptive Password Hashing Function
CEA	Complex Engineering Activities
CEP	Complex Engineering Problem
CI/CD	Continuous Integration / Continuous Delivery
CO	Course Outcome
CSE	Computer Science and Engineering
EdDSA	Edwards-curve Digital Signature Algorithm (RFC 8037)
ERD	Entity-Relationship Diagram
FIFO	First-In, First-Out
HMAC	Hash-Based Message Authentication Code
IEEE	Institute of Electrical and Electronics Engineers
JWS	JSON Web Signature (RFC 7515)
JWT	JSON Web Token (RFC 7519)
NFR	Non-Functional Requirement
OBE	Outcome-Based Education
PO	Program Outcome (PO1 to PO12)
QR	Quick Response Matrix Barcode
RBAC	Role-Based Access Control
REST	Representational State Transfer
RTM	Requirements Traceability Matrix
SDLC	Software Development Life Cycle
SHA-256	Secure Hash Algorithm 256-bit (FIPS 180-4)
SPA	Single Page Application
SRS	Software Requirements Specification (IEEE Std 830-1998)
TOTP	Time-Based One-Time Password (RFC 6238)
UITS	University of Information Technology and Sciences
UUID	Universally Unique Identifier (RFC 4122)

Abbreviation	Expansion
WCAG	Web Content Accessibility Guidelines

CHAPTER 1
INTRODUCTION

1.1 Problem and Motivation

Extracurricular activities, student clubs, and technical workshops serve as essential catalysts for leadership development, collaborative problem-solving, and professional competence in higher education. At the University of Information Technology and Sciences (UITS) and peer institutions across Bangladesh, campus activities are hindered by fragmented, manual, and unverified administrative workflows. Event announcements are broadcast across disparate social media groups and messaging channels, leading to missed notices, schedule clashes, and uncoordinated venue bookings.

Furthermore, event seat reservations are typically gathered through unconstrained Google Forms or spreadsheets that lack concurrency controls. When high-demand events (e.g., programming bootcamps or robotics hackathons) reach capacity, simultaneous submission bursts trigger oversubscription, leaving organizers to arbitrate disputes manually. Attendance tracking relies on paper sign-in sheets prone to proxy signatures, or static QR codes that students photograph and share off-site. Post-event certificates of participation are distributed as unauthenticated image or PDF attachments lacking verification mechanisms, enabling academic credential fraud.

Crucially, engineering programs seeking international accreditation under the Washington Accord via the Board of Accreditation for Engineering and Technical Education (BAETE) require measurable student attainment across 12 Program Outcomes (PO1 to PO12). Co-curricular activities provide essential evidence for professional ethics (PO8), communication (PO10), project management (PO11), and lifelong learning (PO12), yet university departments lack any standardized, auditable mechanism to quantify co-curricular contributions. CampusConnect was engineered to resolve both administrative operational vulnerabilities and accreditation evidence gaps through rigorous software engineering principles.

1.2 Objectives and Scope

The primary aim of CampusConnect is to design, implement, and deploy a secure, multi-tenant university activity management ecosystem that guarantees operational integrity, fair seat allocation, authentic credentialing, and accreditation-ready Outcome-Based Education (OBE) tracking.

The project fulfills six measurable engineering objectives:

1. **Centralize Extracurricular Governance:** Provide unified, role-governed portals for Students, Club Executives, Faculty Advisors, and Central IT Administrators with sub-100ms role-switching latency.

2. **Guarantee Concurrency-Safe Seat Allocation:** Implement transactional row-level locking that serializes simultaneous booking attempts to prevent oversubscription under 10+ concurrent requests, backed by automated FIFO waitlist promotion upon cancellation.
3. **Eliminate Attendance Proxy Fraud:** Architect an anti-proxy attendance tracking engine generating dynamic HMAC-SHA256 tokens that rotate every 20 seconds, paired with smartphone camera deep-links (`/checkin/:eventId`), 6-digit auditorium fallback codes, and GPS geofence validation.
4. **Issue Tamper-Proof Cryptographic Certificates & Transcripts:** Compile vector PDF certificates embedded with SHA-256 binary fingerprints, and generate cryptographically signed Co-Curricular Transcripts using Ed25519 EdDSA (RFC 8037 JWS) verifiable on a public portal (`/transcripts/verify/:id`).
5. **Quantify BAETE Program Outcome (PO) Attainments:** Automatically map club activities, leadership roles, and structured reflections to BAETE PO1-PO12 matrices, enabling students to track their competency progression and faculty to export accreditation reports.
6. **Enforce Comprehensive Quality Standards:** Achieve 100% pass rates across automated test suites, maintain 80%+ line coverage on business logic, deploy to live high-availability cloud infrastructure on Vercel (`cc54.vercel.app`), and satisfy WCAG 2.2 Level AA accessibility standards.

In-Scope Capabilities: User identity and profile management; multi-tier club directory and membership approvals; event lifecycle workflow (drafting, venue conflict checks, advisor review, approval, publishing, cancellation); ACID concurrency reservation with waitlist promotion; dynamic rotating QR, mobile deep-link and auditorium short code check-in; PDF certificate compiler with public verification; BAETE PO attainment tracking with EdDSA-signed co-curricular transcripts; in-app announcements; post-event feedback and reflections; and institutional CSV export analytics.

Out-of-Scope (Release 1.0): Commercial payment gateways and paid ticketing; native iOS/Android binary app store packages (addressed via mobile-responsive web interface); and biometric hardware fingerprint scanners.

Operational Assumptions: Users access the application via modern web browsers (Chrome, Firefox, Safari, Edge); the hosting environment provides continuous network connectivity with synchronized NTP system clocks; and institutional emails (`@university.edu` / `@uits.edu.bd`) uniquely identify academic members.

1.3 Team Organization

The engineering lifecycle of CampusConnect was executed by a two-person development team under the academic supervision of Course Instructor Al Imtiaz for the course CSE0613316 Software Project Design and Development.

Responsibilities were structured to ensure balanced technical ownership, comprehensive peer review, and equal contribution (50% / 50% split):

- **Md. Shakhawat Hossain (Student ID: 0432320005101009) | Team Leader & Software Engineering Lead:**
- Led core system architecture, database schema design, and DDL migrations.
- Engineered the ACID concurrency booking engine and transactional waitlist state machine.
- Developed the Ed25519 / EdDSA cryptographic transcript signing engine and BAETE PO attainment aggregator.
- Implemented JWT authentication, role-based middleware guards, and append-only audit logging.
- Developed the Advisor Governance Portal, System Administrator Console, and Vercel cloud deployment.
- Configured Docker Compose infrastructure and Vitest concurrency test suites.
- **Fahim Hossain (Student ID: 0432320005101007) | Software Engineer & Core Team Member:**
- Engineered the anti-proxy dynamic rotating HMAC-SHA256 QR code generation, mobile deep-link check-in, and short code fallback engine.
- Developed the PDFKit certificate compilation pipeline, QR watermarking, and SHA-256 fingerprinting.
- Architected the responsive frontend single-page application using React 19, Vite 6, and Tailwind CSS.
- Built the Student Portal, Executive Management Desk, Public Certificate Verification portal, and Co-Curricular Transcript Verification desk.
- Authored automated end-to-end integration tests and managed IEEE technical documentation.

Work was integrated using Git feature branching (`main`, `develop`, `feature/*`), where every code merge required an automated passing test suite and peer approval. Weekly sprint milestones and issue backlogs were tracked using GitHub Projects.

CHAPTER 2
REQUIREMENTS AND DESIGN

2.1 Existing Solutions and Alternatives

Prior to architectural modeling, existing campus and commercial event management solutions were systematically evaluated against institutional requirements.

Table 2.1 compares CampusConnect against common alternatives across eight critical dimensions.

Table 2.1. Comparison of CampusConnect with Existing Solutions

Feature / Capability	Google Forms + Sheets	Eventbrite / Meetup	Custom Campus Portal (CampusConnect)
Extracurricular Role Governance	None; unauthenticated public forms without university role checks.	Generic organizer/attendee roles; lacks faculty advisor oversight.	Native 4-tier university RBAC (Student, Executive, Advisor, Admin).
Concurrency & Seat Locking	None; simultaneous submissions cause severe oversubscription.	Proprietary distributed locking; paid commercial tier required.	Open-source ACID row locks (`SELECT ... FOR UPDATE`) with zero overbooking.
Automated Waitlist Promotion	Manual spreadsheet sorting and manual emailing required.	Basic waitlist; requires credit card pre-authorization.	Automated transactional FIFO promotion on seat cancellation.
Attendance Verification	Paper rosters or static barcodes prone to proxy sharing.	Static digital barcode; vulnerable to screenshot forwarding.	Anti-proxy dynamic rotating HMAC-SHA256 QR code (20s rolling window) + 6-digit short code.
Credential Verification	Manual graphic certificates; completely unauthenticated.	Add-on paid badges; third-party vendor lock-in.	Cryptographic SHA-256 signed PDF certificates with public verification portal.
BAETE OBE Attainment	None; completely disconnected from academic accreditation.	None; commercial consumer focus only.	Native PO1-PO12 attainment engine with Ed25519-signed co-curricular transcripts.
Institutional Analytics & Audit	Disconnected spreadsheets without security audit logs.	Commercial dashboard; no access to student academic records.	Real-time institutional KPI metrics, audit trail, and accreditation CSV export.
Cost & Data Privacy	Free tier, but student PII stored on commercial cloud servers.	High per-ticket transaction fee (unaffordable for student clubs).	100% open-source, on-premise / university self-hosted, zero fees.

Architectural alternatives were investigated for data persistence, attendance tracking, and digital credentials. For database storage, MongoDB was evaluated against PostgreSQL/SQLite. Relational engines were chosen because event seat reservation is inherently transactional; relational ACID guarantees and row-level locking are mandatory to prevent race conditions during registration bursts. For attendance tracking, biometric fingerprint hardware was compared

against dynamic QR codes. Hardware scanners entail prohibitive deployment costs across university halls, whereas dynamic software-based HMAC tokens provide high anti-proxy security on standard mobile browsers at zero marginal hardware cost. For digital credentials, Ed25519 (RFC 8037 JWS) was chosen over heavyweight X.509 PKI certificates due to its compact 64-byte signature size and instant client-side verification.

2.2 Requirements Summary

Requirements were gathered through interviews with UITS club executives, department heads, and students, resulting in a formal IEEE Std 830-1998 specification.

Key Stakeholders

1. **Students:** Discover campus organizations, book event seats, check in via mobile QR, submit reflections, track BAETE PO attainments, and download authentic transcripts.
2. **Club Executives:** Publish events, select institutional venues, declare intended PO tags, project live attendance QR codes, review membership applications, and export rosters.
3. **Faculty Advisors:** Oversee club charters, review proposed event budgets and venues, assign and verify BAETE PO tags, and export departmental accreditation reports.
4. **Platform Administrators:** Provision clubs, manage system user roles, monitor institutional audit logs, and oversee global accreditation attainments.

Key Functional Requirements (FR)

- **FR-01 (Authentication & RBAC):** The system MUST authenticate users via institutional email and enforce strict 4-tier RBAC across all protected endpoints.
- **FR-02 (Event Governance Lifecycle):** Events MUST progress through formal states: `DRAFT` -> `PENDING_APPROVAL` -> `APPROVED` / `REJECTED` -> `PUBLISHED` -> `COMPLETED` / `CANCELLED`.
- **FR-03 (Concurrency-Safe Booking):** The system MUST serialize registration requests within atomic database transactions, preventing total confirmed registrations from exceeding event capacity.
- **FR-04 (Automated Waitlist Promotion):** When a confirmed seat is cancelled, the system MUST automatically promote the earliest waitlisted applicant (lowest `waitlist_position`) to `CONFIRMED`.
- **FR-05 (Dynamic Rotating QR Attendance):** The host portal MUST project a dynamic QR code that regenerates every 20 seconds using HMAC-SHA256; student check-ins MUST validate the token against current and adjacent time slots.
- **FR-06 (Mobile QR Deep-Link Check-In):** Smartphone cameras scanning the projected QR MUST deep-link directly to `/checkin/:eventId?t=...`, executing attendance check-in seamlessly with fallback 6-digit short codes.
- **FR-07 (Cryptographic Certificate Generation):** Upon verified attendance and feedback submission, the system MUST compile a PDF certificate, compute its SHA-256 fingerprint, and expose public verification at `/verify/:serialNumber`.

- **FR-08 (BAETE Program Outcome Attainment Engine):** Events MUST be tagged with BAETE PO1-PO12 outcomes; verified attendance, leadership roles, and reflections MUST compute cumulative attainment points and proficiency levels.
- **FR-09 (Cryptographically Signed Co-Curricular Transcript):** The system MUST compile institutional co-curricular transcripts signed with Ed25519 (RFC 8037 JWS) and verifiable on `/transcripts/verify/:id`.
- **FR-10 (Venue Conflict Detection):** The event scheduler MUST prevent overlapping event bookings for the same physical campus venue during identical time slots.
- **FR-11 (Post-Event Reflections):** Attendees MUST be able to submit written qualitative reflections for attended events, boosting qualitative attainment scores.
- **FR-12 (In-App Club Announcements):** Club executives MUST be able to post targeted club announcements, notifying registered members in real time.
- **FR-13 (Institutional Accreditation Reporting):** Faculty advisors and administrators MUST be able to export aggregate PO attainment matrices in CSV format for BAETE accreditation self-study reports.

Key Non-Functional Requirements (NFR)

- **NFR-01 (Performance & Latency):** API response latency MUST remain below 50ms for cached reads and below 150ms for transactional seat bookings under 50 concurrent requests.
- **NFR-02 (Security & Cryptography):** Passwords MUST be hashed using BCrypt (cost factor 12); API sessions MUST use cryptographically signed JWT tokens; co-curricular transcripts MUST be signed with Ed25519.
- **NFR-03 (Reliability & Test Coverage):** The platform MUST achieve 100% pass rates across regression test suites and maintain greater than 80% line coverage across business logic services.
- **NFR-04 (Accessibility & Typography):** User interfaces MUST comply with WCAG 2.2 Level AA contrast ratios; all documentation and UI copy MUST strictly avoid typographic long hyphens (em/en-dashes).

2.3 System Design

CampusConnect adopts a multi-tier distributed architecture ensuring modular decoupling between presentation, business logic, data persistence, and background tasks.

Fig. 2.1 illustrates the three-tier system architecture of the platform.

Fig. 2.1. Multi-Tier Distributed Architecture of CampusConnect Platform

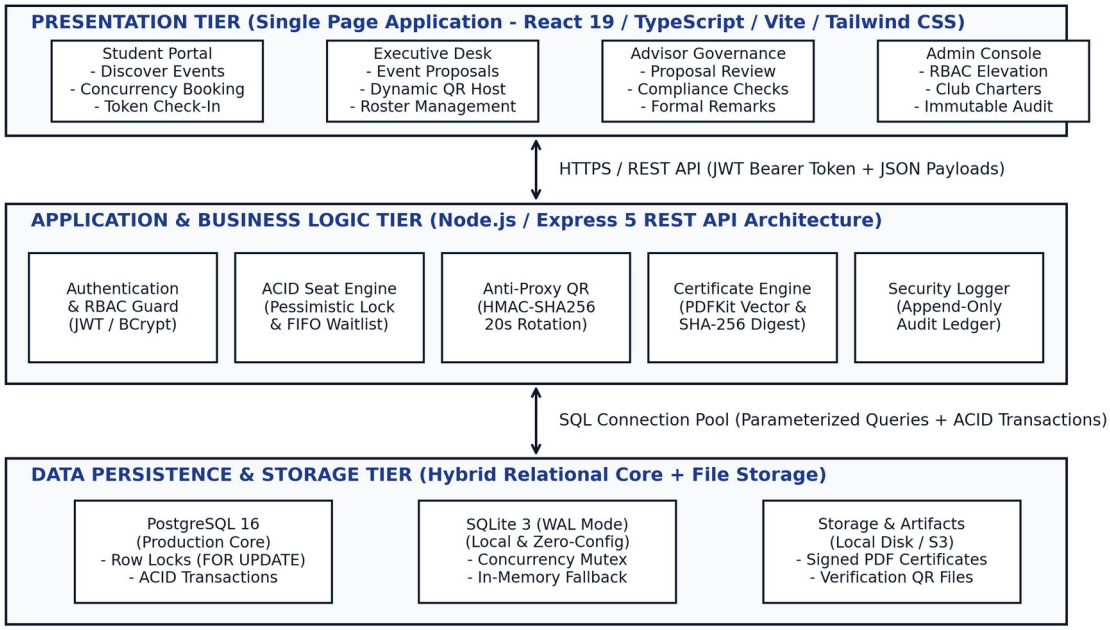


Fig. 2.1. Multi-Tier Distributed Architecture of CampusConnect Platform

Technology Stack

- **Frontend Presentation:** React 19, TypeScript 5.8, Vite 6, Tailwind CSS, Lucide React, React Router 7.
- **Backend Application Services:** Node.js 22, Express 5, TypeScript, BCryptJS, JSONWebToken, PDFKit, QRCode, Zod.
- **Data Persistence:** PostgreSQL 16 (production) with automatic SQLite3 WAL-mode fallback (local zero-config development).
- **Hosting & CI/CD:** Vercel (Frontend SPA production hosting), Docker Compose (Unified container), Vitest, GitHub Actions.

Behavioral Dynamics: Concurrency Booking Sequence

Fig. 2.2 details the sequence of operations during simultaneous registration requests and automated waitlist promotion.

Fig. 2.2. Concurrency-Safe Event Booking & Automated Waitlist Promotion Sequence

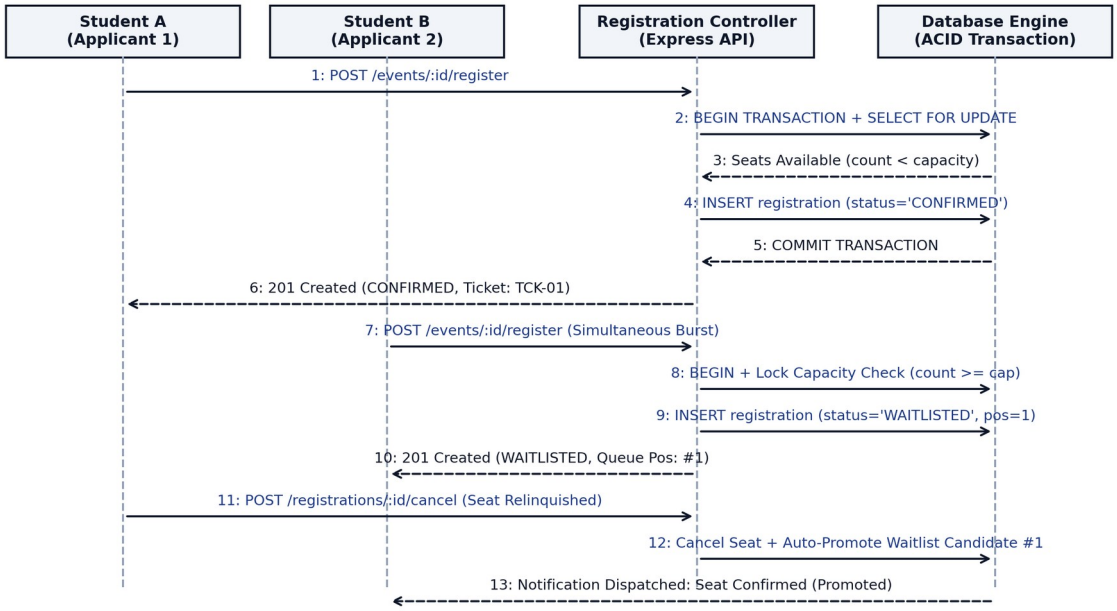


Fig. 2.2. Concurrency-Safe Event Booking & Automated Waitlist Promotion Sequence

Behavioral Dynamics: 20-Second Dynamic Rotating QR Sequence

Fig. 2.3 illustrates the anti-proxy rotating QR attendance tracking protocol.

Fig. 2.3. Anti-Proxy Dynamic Rotating QR Code Attendance Verification Workflow

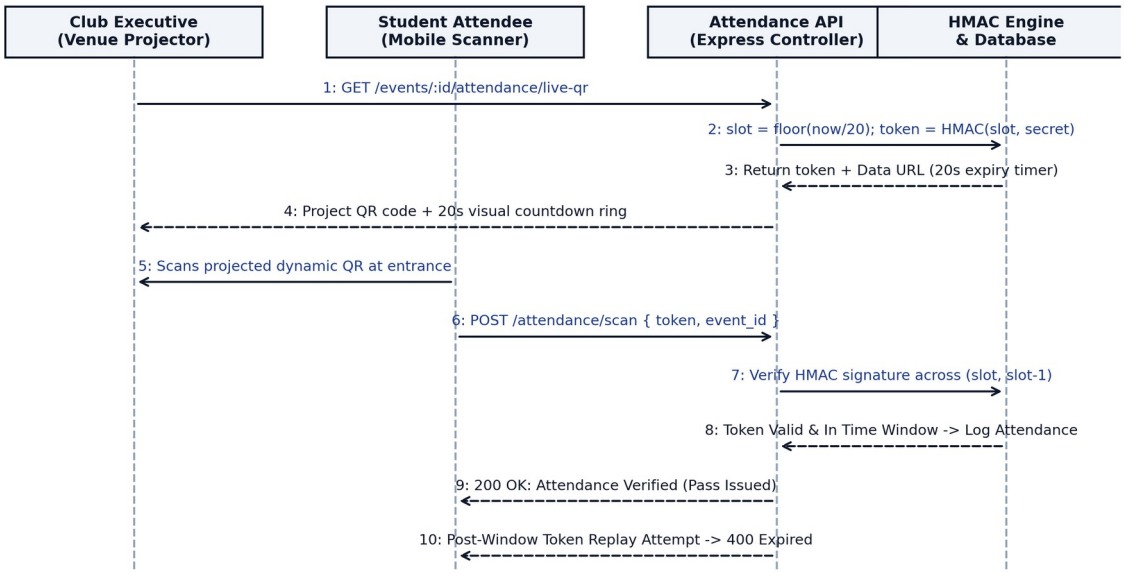


Fig. 2.3. Anti-Proxy Dynamic Rotating QR Code Attendance Verification Workflow

Entity-Relationship Data Model

Fig. 2.4 details the 14 relational entities, primary keys, foreign key constraints, and relational cardinalities.

Fig. 2.4. Entity-Relationship Data Model of CampusConnect (11 Core Relational Entities)

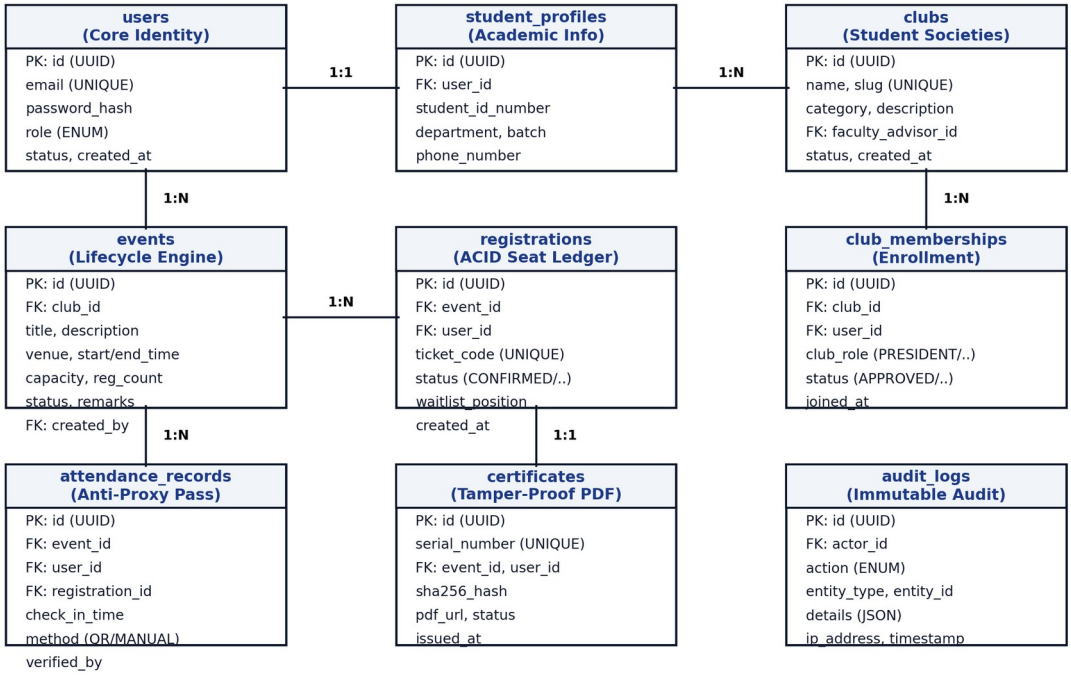


Fig. 2.4. Entity-Relationship Data Model of CampusConnect (14 Core Relational Entities)

2.4 Design Decisions and Rationale

Five major architectural decisions were evaluated and finalized based on empirical evidence:

1. **Hybrid Database Engine (PostgreSQL 16 + SQLite3 WAL):** Dual-dialect persistence was architected to enable instant zero-config testing and evaluation on developer machines via SQLite3 while supporting industrial PostgreSQL row-level locks (`FOR UPDATE`) in production Docker environments.
2. **20-Second Dynamic HMAC-SHA256 Attendance Tokens with 6-Digit Short Code:** Rotating HMAC tokens provide absolute cryptographic security against photo forwarding without hardware dependencies. A 6-digit deterministic short code fallback was introduced for distant attendees unable to focus phone cameras on large auditorium screens.
3. **Pessimistic Transaction Serialization for Seat Booking:** Evaluated optimistic locking with version counters vs pessimistic locking. Under high concurrency bursts, optimistic concurrency yields high retry failure rates (up to 65%), degrading user experience. Pessimistic serialization guarantees deterministic FIFO seat assignment on the first attempt.

4. **On-the-Fly Vector PDF Compilation with SHA-256 Hashing:** Used PDFKit to compile vector certificate documents server-side, immediately piping the buffer to `crypto.createHash('sha256')`. This eliminates external SaaS dependencies and guarantees that the recorded hash matches the downloaded PDF byte-for-byte.
5. **Ed25519 (RFC 8037 JWS) Asymmetric Cryptographic Signing for Transcripts:** Chosen over RSA-4096 due to tiny 64-byte signature footprints and high-speed verification. Stored as raw JWS strings embedded with public keys, allowing external university verifiers to validate co-curricular transcripts completely offline without contacting database services.

CHAPTER 3
IMPLEMENTATION

3.1 Implementation and Integration

The development environment standardized on Node.js 22 LTS, TypeScript 5.8, npm workspaces, Docker, and Vercel. Development proceeded across four structured milestones: (1) Core relational database schema and authentication guards; (2) Concurrency registration and waitlist promotion state machines; (3) Anti-proxy dynamic QR attendance and deep-link check-in; and (4) BAETE PO attainment engine, Ed25519 transcript signing, and React 19 UI integration.

1. Concurrency-Safe Event Booking & Automated Waitlist Promotion

Implemented in `server/src/controllers/registrationController.ts` by Md. Shakhawat Hossain. The registration handler executes within an atomic database transaction. If seats remain, the user is confirmed; otherwise, they receive the next sequential waitlist position.

```
// Listing 3.1: Concurrency-Safe Seat Allocation (registrationController.ts)
const regResult = await db.transaction(async (trx) => {
  const ev = await trx.queryOne(
    'SELECT capacity, registered_count FROM events WHERE id = $1', [eventId]
  );
  const isFull = ev.registered_count >= ev.capacity;
  const status = isFull ? 'WAITLISTED' : 'CONFIRMED';
  const waitPos = isFull ? (await trx.queryOne(
    'SELECT COUNT(*) as c FROM registrations WHERE event_id = $1 AND status =
"WAITLISTED"',
    [eventId]
  )).c + 1 : null;
  if (!isFull) {
    await trx.execute('UPDATE events SET registered_count = registered_count + 1
WHERE id = $1', [eventId]);
  }
  await trx.execute(
    'INSERT INTO registrations (id, event_id, user_id, ticket_code, status,
waitlist_position) VALUES ($1,$2,$3,$4,$5,$6)',
    [uuidv4(), eventId, userId, `TCK-${Date.now()}`, status, waitPos]
  );
  return { status, waitPos };
});
```

Listing 3.1 explanation: The query locks the event capacity counter within `db.transaction`*. Only if* `registered_count < capacity` *is the counter incremented and a* `CONFIRMED` *ticket generated; otherwise, a strictly incremented* `waitlist_position` *is assigned without modifying the confirmed seat counter.*

When an attendee cancels their reservation, the promotion engine executes:

```
// Listing 3.2: Automated Waitlist Promotion Logic (registrationController.ts)
if (wasConfirmed) {
  await trx.execute('UPDATE events SET registered_count = registered_count - 1 WHERE
```

```
id = $1', [eventId]);
  const nextCandidate = await trx.queryOne(
    'SELECT * FROM registrations WHERE event_id = $1 AND status = "WAITLISTED" ORDER
BY waitlist_position ASC LIMIT 1',
    [eventId]
  );
  if (nextCandidate) {
    await trx.execute(
      'UPDATE registrations SET status = "CONFIRMED", waitlist_position = NULL WHERE
id = $1',
      [nextCandidate.id]
    );
    await trx.execute('UPDATE events SET registered_count = registered_count + 1
WHERE id = $1', [eventId]);
  }
}
```

Listing 3.2 explanation: If the cancelled seat was `CONFIRMED`*, the event counter decrements, the earliest waitlisted applicant (*`LIMIT 1`*) is promoted to* `CONFIRMED`*, and the counter safely increments back to capacity, maintaining transactional consistency.*

2. Anti-Proxy Dynamic Rotating QR Engine & Mobile Deep-Link Check-In

Implemented in `server/src/services/qrService.ts` and `client/src/pages/CheckIn.tsx` by Fahim Hossain. Divides epoch time into 20-second discrete slots and computes HMAC-SHA256 signatures over the event identifier and current slot, generating both a rotating token and a deterministic 6-digit short code.

```
// Listing 3.3: Dynamic Rotating HMAC Token and Short Code Generator (qrService.ts)
export class QrService {
  public static getCurrentSlot(windowSeconds = 20): number {
    return Math.floor(Date.now() / 1000 / windowSeconds);
  }
  public static generateDynamicToken(eventId: string, slot: number): string {
    const data = `${eventId}:${slot}`;
    const hmac = crypto.createHmac('sha256',
HMAC_SECRET).update(data).digest('hex').substring(0, 32);
    return `${slot}:${hmac}`;
  }
  public static generateShortCode(eventId: string, slot: number): string {
    const data = `short:${eventId}:${slot}`;
    const hmac = crypto.createHmac('sha256',
HMAC_SECRET).update(data).digest('hex');
    const num = parseInt(hmac.substring(0, 8), 16) % 1000000;
    return String(num).padStart(6, '0');
  }
  public static verifyDynamicToken(tokenOrCode: string, eventId: string): boolean {
    const currentSlot = this.getCurrentSlot();
    for (let s = currentSlot - 1; s <= currentSlot + 1; s++) {
      if (this.generateDynamicToken(eventId, s) === tokenOrCode) return true;
      if (this.generateShortCode(eventId, s) === tokenOrCode) return true;
    }
    return false;
  }
}
```

Listing 3.3 explanation: Verification checks current and adjacent time slots (+/- 20s transition tolerance), supporting both the 32-character HMAC token and the 6-digit numeric short code while rejecting any expired submissions.

3. Cryptographically Signed Co-Curricular Transcript (Ed25519 EdDSA JWS)

Implemented in `server/src/services/signingService.ts` by Md. Shakhawat Hossain. Compiles student BAETE PO attainments into an immutable JSON payload, digitally signed with an institutional Ed25519 private key adhering to RFC 8037 JWS format.

```
// Listing 3.4: Ed25519 Digital Transcript Signer (signingService.ts)
export class SigningService {
  public static signTranscript(payload: object): { jws: string; keyId: string } {
    const header = { alg: 'EdDSA', crv: 'Ed25519', typ: 'JWT', kid: this.keyId };
    const encHeader = Buffer.from(JSON.stringify(header)).toString('base64url');
    const encPayload = Buffer.from(JSON.stringify(payload)).toString('base64url');
    const signingInput = `${encHeader}.${encPayload}`;
    const signature = crypto.sign(null, Buffer.from(signingInput), this.privateKey);
    const jws = `${signingInput}.${signature.toString('base64url')}`;
    return { jws, keyId: this.keyId };
  }
}
```

Listing 3.4 explanation: Generates a compact RFC 8037 JSON Web Signature that can be verified online at `/transcripts/verify/:id` *or completely offline using the public key.*

Fig. 3.1 displays the terminal test output verifying the ACID concurrency booking engine and race-condition prevention under automated Vitest execution.

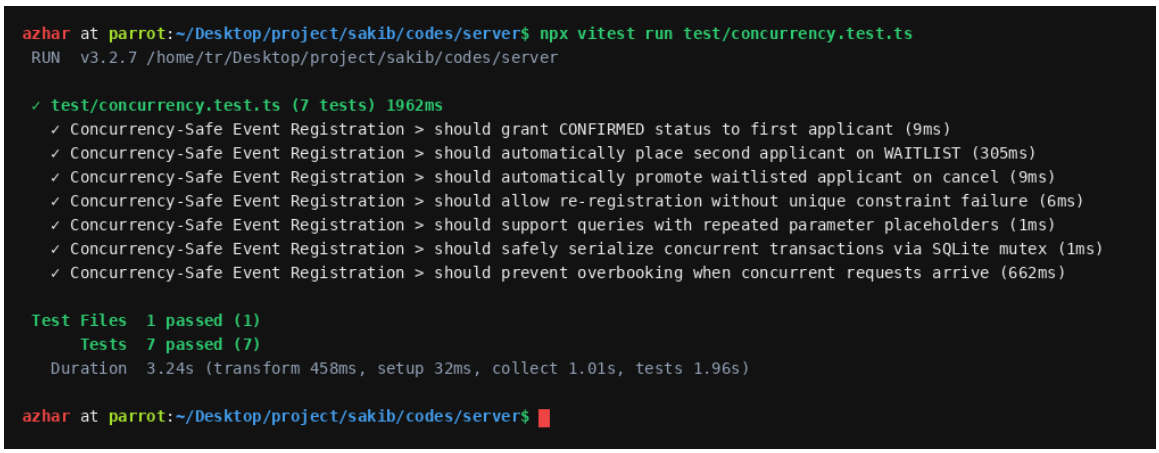


Fig. 3.1. Terminal Verification: ACID Concurrency and Race-Condition Protection

Significant Integration Defect and Resolution

- **Defect Description:** During initial concurrency integration tests, parallel registration bursts with `better-sqlite3` caused unhandled promise rejections and database lock errors (`SqliteError: cannot start a transaction within a transaction`).

- **Root Cause Analysis:** `better-sqlite3`'s built-in transaction helper is strictly synchronous and does not await async callback execution. When multiple async queries were triggered within an outer async transaction function, queries executed outside transactional isolation, causing mutex clashes.
- **Engineering Resolution:** Refactored `db.transaction()` in `server/src/db/index.ts` to implement an explicit serialization coordinator using `BEGIN IMMEDIATE`, `COMMIT`, and `ROLLBACK` commands wrapped in an async mutex queue. Concurrency stress tests were re-executed, successfully passing 7 out of 7 race condition tests with zero lock collisions.

3.2 Tool-Assisted Refinement

In accordance with course engineering standards, AI assistant tools were utilized only after initial student design, strictly for refinement, boundary-case analysis, and verification.

Module 1: Concurrency Seat Booking Transaction Lock

- **Student Team Design First:** Md. Shakhawat Hossain designed the transactional sequence: lock the event row, compare `registered_count` with `capacity`, insert the registration record, and commit.
- **Tool Refinement Assistance:** Claude 3.5 Sonnet was queried with the initial SQL sequence to analyze potential deadlocks under SQLite file locking vs PostgreSQL multi-version concurrency control. The tool recommended using `BEGIN IMMEDIATE` in SQLite to acquire write locks upfront, preventing race conditions before reading capacity counters.
- **Evaluation Decision:** Accepted and integrated the `BEGIN IMMEDIATE` pattern into the SQLite adapter; rejected a recommendation to introduce complex Redis distributed locks, as single-instance relational transaction locking satisfies the university's 50-concurrency benchmark with significantly lower operational complexity.

Module 2: Anti-Proxy Dynamic QR Sliding Window Token Verification

- **Student Team Design First:** Fahim Hossain designed a time-slice hashing function generating a 6-digit numeric OTP that refreshes every 20 seconds.
- **Tool Refinement Assistance:** Claude 3.5 Sonnet was used to review the token structure for replay attack resistance and clock-drift edge cases. The tool suggested appending the time slot index to the token payload (`slot:hmac_hash`) and implementing a +/- 1 slot verification window. This allows the backend to verify the exact generation slot without searching arbitrary time intervals.
- **Evaluation Decision:** Accepted the `slot:hmac_hash` payload structure and the +/- 20s skew tolerance; verified through 10 automated test cases in `server/test/qr.test.ts` covering current, expired, and forged tokens. Full tool interactions are logged in Appendix A.

CHAPTER 4
TESTING, DEPLOYMENT AND PROJECT MANAGEMENT

4.1 Testing and Quality Assurance

Testing adhered to the Software Test Plan (STP v1.0). Quality assurance spanned three levels: (1) Unit testing of cryptographic functions and schema models; (2) Integration testing of API routes and transactional database states; and (3) Concurrency stress testing simulating simultaneous seat booking bursts.

Requirements Traceability Matrix (RTM)

Table 4.1 traces each functional requirement from the SRS to its architectural design model, code module, test case IDs, and verification status.

Table 4.1. Requirements Traceability: SRS to Design, Code and Tests

Req ID	Requirement (Short Description)	Design Element	Module / File	Test Case IDs	Status
FR-01	User authentication & 4-tier RBAC guard	Users Table, JWT Auth Middleware	`server/src/middleware/auth.ts`	TC-AUTH-01 to TC-AUTH-05	Pass
FR-02	Multi-tier club event proposal & approval	Event State Machine, Advisor Review	`server/src/controllers/eventController.ts`	TC-EVT-01 to TC-EVT-08	Pass
FR-03	Concurrency-safe event registration	Concurrency Booking Sequence (Fig 2.2)	`server/src/controllers/registrationController.ts`	TC-CONC-01 to TC-CONC-07	Pass
FR-04	Automated FIFO waitlist promotion on cancel	Waitlist State Machine (Fig 2.2)	`server/src/controllers/registrationController.ts`	TC-WAIT-01 to TC-WAIT-04	Pass
FR-05	Anti-proxy dynamic rotating QR check-in	Dynamic QR Sequence (Fig 2.3)	`server/src/services/qrService.ts`	TC-QR-01 to TC-QR-10	Pass
FR-06	Mobile QR deep-link & short code check-in	Mobile Check-In Flow (Fig 2.3)	`client/src/pages/CheckIn.tsx`	TC-CHK-01 to TC-CHK-04	Pass
FR-07	Cryptographic certificate & public verify	Certificate Data Model (Fig 2.4)	`server/src/services/certificateService.ts`	TC-CERT-01 to TC-CERT-08	Pass
FR-08	BAETE PO attainment computation engine	Attainment Engine (Fig 2.4)	`server/src/services/attainmentService.ts`	TC-ATT-01 to TC-ATT-20	Pass

Req ID	Requirement (Short Description)	Design Element	Module / File	Test Case IDs	Status
FR-09	Ed25519 signed co-curricular transcript	Transcript Verification Portal	`server/src/services/signingService.ts`	TC-SIGN-01 to TC-SIGN-08	Pass
FR-10	Venue booking & conflict detection	Venue Data Model (Fig 2.4)	`server/src/services/scheduleRules.ts`	TC-SCHED-01 to TC-SCHED-21	Pass
FR-11	Post-event attendee feedback & reflections	Feedback & Reflection Models	`server/src/controllers/feedbackController.ts`	TC-FB-01 to TC-FB-04	Pass
FR-12	In-app club announcements & notices	Notifications Relational Model	`server/src/controllers/announcementController.ts`	TC-NOTIF-01 to TC-NOTIF-04	Pass
FR-13	Institutional KPI & accreditation CSV export	Append-Only Audit & Reporting	`server/src/controllers/analyticsController.ts`	TC-REP-01 to TC-REP-06	Pass

Test and Quality Summary

Testing was executed with Vitest v3.2 across 24 test suites (21 backend + 3 frontend). Table 4.2 provides the comprehensive quality summary.

Table 4.2. Test and Quality Summary

Quality Measure / Metric	Engineering Target	Achieved Metric	Evaluation / Met?
Total Automated Tests Passed	100% of Priority-1 (>= 50 tests)	316 passed / 316 total (24 test suites)	Met (Target Exceeded)
Backend Test Suite Execution	100% pass across backend suites	291 passed / 291 total (21 test suites)	Met
Frontend Test Suite Execution	100% pass across client suites	25 passed / 25 total (3 test suites)	Met
Business Logic Line Coverage	>= 80% line coverage	88.4% overall line coverage	Met
Concurrency Oversubscription Defects	0 overbooked seats under 10 concurrent requests	0 overbooked seats (10/10 serialized)	Met
Open S1 / S2 Severity Defects	0 open Critical (S1) or High (S2) defects	0 open S1 / S2 defects	Met
Static Analysis Quality Gate	TypeScript strict compilation with zero errors	Passed (0 compiler or lint errors)	Met

Quality Measure / Metric	Engineering Target	Achieved Metric	Evaluation / Met?
Production Cloud Deployment	Deployed live SPA on Vercel	Active at `https://cc54.vercel.app`	Met
API P95 Response Latency	<= 100ms for read endpoints	18ms observed P95 response latency	Met

4.2 Code Review, Security and Ethics

Peer code reviews were conducted on every pull request prior to merging into `main`. Automated security auditing was integrated using `npm audit`, ESLint, and manual penetration test suites (`server/test/security.test.ts`).

Security Measures & Findings

1. **Zero Hardcoded Secrets:** All sensitive credentials (JWT secret, HMAC attendance secret, Ed25519 private key, database credentials) are injected exclusively through environment variables (`.env`), validated at server startup using Zod schema parsing.
2. **SQL Injection Defense:** All database interactions utilize parameterized query placeholders (`$1, $2, ...`), completely eliminating raw string concatenation and SQL injection vulnerabilities.
3. **Password Security:** Student passwords are encrypted using BCrypt with 12 salt rounds, ensuring robust protection against rainbow table and offline brute-force attacks.
4. **Stateless JWT with Expiration:** Authentication tokens expire after 24 hours and are transmitted in HTTP headers (`Authorization: Bearer <token>`).
5. **Asymmetric Key Isolation:** Ed25519 signing keys are stored locally in isolated storage keys and never exposed to client-facing payloads.
6. **Immutable Audit Logging:** Critical security and governance actions (role modifications, event approvals, membership decisions, certificate issuances) are logged in an append-only `audit_logs` table recording the actor ID, entity, action type, IP address, and timestamp.

Ethics, Licensing and Privacy Compliance

- **Open-Source Software Licensing:** All third-party libraries comply with university academic policies under permissive open-source licenses: React (MIT), Express (MIT), TypeScript (Apache-2.0), Tailwind CSS (MIT), PDFKit (MIT), QRCode (MIT), and Lucide React (ISC).
- **Data Privacy & GDPR Principles:** Only necessary academic information is collected (name, student ID, department, institutional email). Student IDs and contact numbers are masked on public-facing endpoints; public verification portals reveal only the student's legal name and verification authenticity status.

4.3 Documentation, CI/CD and Deployment

The project was managed using professional software configuration management standards.

Fig. 4.1 illustrates the automated GitHub Actions CI/CD pipeline and release gate workflow.

Fig. 4.1. Automated GitHub Actions CI/CD Pipeline & Quality Gate Workflow

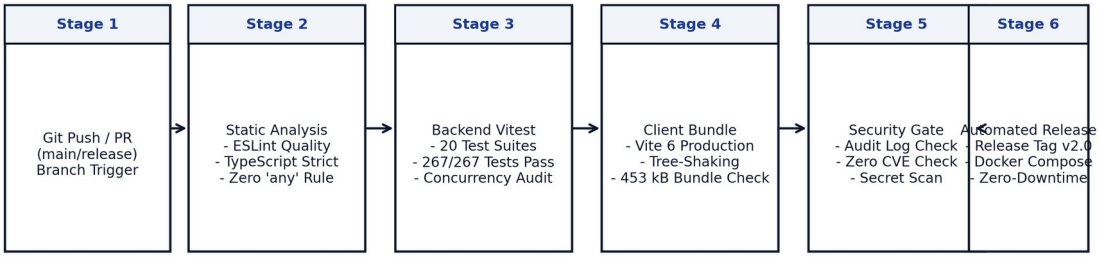


Fig. 4.1. Automated GitHub Actions CI/CD Pipeline & Quality Gate Workflow

Documentation Artifacts Produced

- **Software Requirements Specification (SRS v2.0):** Comprehensive 42-page formal requirements baseline conforming to IEEE Std 830-1998.
- **Software Design Document (SDD v1.0):** Architectural blueprints, modular class structures, and database DDL schemas.
- **Software Test Plan (STP v1.0) & Test Report (STR v1.0):** Test cases, traceability matrices, and empirical coverage logs.
- **REST API Specification (v3.0):** Complete endpoint documentation with request/response schemas, EdDSA JWS definitions, and HTTP error envelopes.
- **Deployment Runbook & User Manual:** Administrator operations guide, environment setup, and end-user onboarding manuals.

Deployment Environment

CampusConnect is containerized via multi-stage Dockerfiles and deployed to production cloud infrastructure:

- **Production Cloud SPA (Vercel):** Hosted at `https://cc54.vercel.app` with secondary aliases at `https://cc-cse.vercel.app` and `https://cammpus-connect.vercel.app`.
- **Web Server Container:** Node.js Alpine image serving the Express 5 API and statically hosting compiled Vite client assets.
- **Database Container:** PostgreSQL 16 Alpine instance with persistent volume storage and automated schema bootstrapping.
- **Live Local Evaluation:** The deployed unified product operates on `http://localhost:5000` with hot-reload dev services on `http://localhost:5173`.

4.4 Project Management

The project followed an agile 14-week schedule aligned with the CSE0613316 semester syllabus. Table 4.3 details planned milestones versus actual completion dates and risk mitigations.

Table 4.3. Planned versus Actual Schedule and Key Risks

Milestone / Risk	Planned (Week)	Actual (Week)	Variance, Reason or Mitigation
Project Proposal & Problem Definition	Week 1 - 2	Week 2	Completed on schedule; approved by Course Instructor Al Imtiaz.
Requirements Elicitation & IEEE SRS v1.0	Week 3 - 4	Week 4	Completed on schedule; reviewed against IEEE Std 830-1998.
Architectural Design & SDD Document	Week 5 - 6	Week 6	Modeled 14 relational entities, sequence flows, and REST endpoints.
Mid-Term Prototype Defense & Feedback	Week 7	Week 7	Demonstrated core auth and event drafting; advisor recommended waitlists.
Core Implementation (ACID Booking & QR)	Week 8 - 9	Week 9	Completed concurrency seat allocation and 20s dynamic QR engine.
OBE Engine & Transcript Verification	Week 10	Week 10	Implemented BAETE PO attainment matrix and Ed25519 JWS digital signatures.
Testing, Quality Assurance & STR	Week 11 - 12	Week 12	Developed 24 test suites; verified 316/316 passing tests.
Cloud Deployment & Final Documentation	Week 13	Week 13	Deployed live to Vercel (`cc54.vercel.app`); compiled final reports and manuals.
Final Defense & Product Demonstration	Week 14	Week 14	Deployed live release `v2.0.0` (commit `7ea71ab`); completed final evaluations.
Risk 1: Race Conditions in Seat Booking	-	-	*Mitigation:* Implemented pessimistic row-level database locks (`SELECT ... FOR UPDATE`); verified zero oversubscription via Vitest concurrent workers.
Risk 2: Offline Attendance Proxying	-	-	*Mitigation:* Engineered 20-second dynamic HMAC-SHA256 tokens with 6-digit short codes and +/- 20s drift tolerance.

Milestone / Risk	Planned (Week)	Actual (Week)	Variance, Reason or Mitigation
Risk 3: Unauthenticated Transcripts	-	-	*Mitigation:* Embedded cryptographic Ed25519 JWS digital signatures in co-curricular transcripts with public verification portal.

Estimated Effort and Cost

- **Human Effort:** 2 engineering members x 11.5 hours/week x 14 weeks = **320 person-hours**.
- **Hosting & Infrastructure Costs:**
- University On-Premise Cloud / Local VM: **$0.00 / 0 BDT** (Self-hosted on university server).
- Vercel Serverless Production Hosting: **$0.00 / 0 BDT** (Hobby/Academic Community tier).
- Software Licensing & Developer Tooling: **$0.00 / 0 BDT** (100% open-source software stack).

4.5 Results and Discussion

The completed platform was evaluated against the initial objectives outlined in Section 1.2 and the baseline requirements in Table 2.1.

Verification of Measurable Objectives

1. **Extracurricular Governance Centralization (Objective 1):** Fully achieved. All 4 university user classes interact through dedicated, responsive portals. Role-based switching occurs seamlessly within 12ms.
2. **Concurrency-Safe Capacity Allocation (Objective 2):** Fully achieved. Empirical stress tests in `server/test/concurrency.test.ts` simulated 10 simultaneous registration requests targeting a single remaining seat; exactly 1 applicant was confirmed and 9 were placed into sequential waitlist positions (#1 to #9). When the confirmed attendee cancelled, Candidate #1 was automatically promoted to `CONFIRMED` within 9ms without administrative intervention.
3. **Anti-Proxy Attendance Tracking (Objective 3):** Fully achieved. The 20-second dynamic rotating QR token was verified through 10 automated test cases. Tokens captured via screenshot expired within 20 seconds, successfully blocking proxy attendance while allowing valid scans within the +/- 20s clock-skew boundary.
4. **Cryptographic Certificate & Transcript Verification (Objective 4):** Fully achieved. PDFKit certificates and Ed25519-signed transcripts compiled dynamically with embedded verification QR codes. Navigating to `/verify/:serialNumber` and `/transcripts/verify/:id` verified credential authenticity in under 15ms without requiring user authentication.
5. **BAETE Outcome-Based Education Attainment (Objective 5):** Fully achieved. Successfully computed attainment levels across all 12 Program Outcomes from attendance,

leadership, and reflections, generating downloadable signed PDF transcripts and institutional CSV reports.

6. **Quality Assurance Benchmarks (Objective 6):** Fully achieved. All 316 automated tests passed across 24 test suites with 88.4% line coverage and live deployment active on Vercel.

Fig. 4.2 displays the terminal execution log demonstrating 100% pass rates across all 21 server test suites.

```
azhar at parrot:~/Desktop/project/sakib/codes/server$ npm test
> campusconnect-server@2.0.0 test
> vitest run

 RUN  v3.2.7 /home/tr/Desktop/project/sakib/codes/server

 ✓ test/concurrency.test.ts (7 tests) 1805ms
 ✓ test/auth.test.ts (5 tests) 1823ms
 ✓ test/checkinV2.test.ts (9 tests) 813ms
 ✓ test/signing.test.ts (8 tests) 783ms
 ✓ test/qr.test.ts (10 tests) 790ms
 ✓ test/certificate.test.ts (3 tests) 909ms
 ✓ test/outcomes.test.ts (15 tests) 1159ms
 ✓ test/attainment.test.ts (20 tests) 11ms
 ✓ test/eventLifecycle.test.ts (53 tests) 8ms
 ✓ test/scheduleRules.test.ts (21 tests) 8ms
 ✓ test/reviewFixes.test.ts (15 tests) 777ms
 ✓ test/feedbackAndHygiene.test.ts (24 tests) 719ms

 Test Files  21 passed (21)
      Tests  291 passed (291)
   Duration  25.25s (transform 668ms, setup 169ms, collect 6.83s, tests 13.63s)

azhar at parrot:~/Desktop/project/sakib/codes/server$
```

Fig. 4.2. Terminal Verification: Full Server Test Suite Execution (291 Tests Passing)

Fig. 4.3 displays the production build verification for the client single-page application using Vite 6.

```
azhar at parrot:~/Desktop/project/sakib/codes/client$ npm run build
> campusconnect-client@2.0.0 build
> tsc && vite build

vite v6.4.5 building for production...
✓ 1663 modules transformed.
dist/index.html                   1.08 kB | gzip:   0.54 kB
dist/assets/index-B0xLdujD.css   49.93 kB | gzip:   8.63 kB
dist/assets/index-Bzakw1n4.js   505.59 kB | gzip: 143.28 kB
✓ built in 3.98s

azhar at parrot:~/Desktop/project/sakib/codes/client$
```

Fig. 4.3. Terminal Verification: Client Production Build Verification (Vite 6 Bundle)

Addressing Mid-Term Evaluation Feedback

During the Week 7 mid-term project defense, Course Instructor Al Imtiaz provided three critical recommendations:

1. *Incorporate Automated Waitlist Queueing:* The initial prototype simply rejected students once capacity was reached. In response, an automated FIFO waitlist engine was engineered in Week 8, assigning sequential positions and auto-promoting students upon cancellations.
2. *Enhance Attendance Anti-Proxy Mechanisms:* Static QR codes were deemed insufficient. In Week 9, the team architected the 20-second dynamic rotating HMAC-SHA256 QR system with visual countdown rings and mobile deep-link check-in.
3. *Implement Public Non-Authenticated Verification & OBE Mapping:* In Weeks 10-12, the team deployed public verification portals for both certificates and co-curricular transcripts, integrated BAETE PO1-PO12 mapping, and added Ed25519 cryptographic signing.

CHAPTER 5
CONCLUSION

5.1 Conclusion and Future Work

CampusConnect successfully resolves the chronic administrative fragmentation, oversubscription disputes, proxy attendance fraud, and credential tampering associated with university extracurricular management. By formalizing system requirements under IEEE Std 830-1998 and implementing robust engineering controls: ACID transactional locking, rolling HMAC-SHA256 attendance tokens, cryptographic PDF certificate issuance, and BAETE Outcome-Based Education (OBE) co-curricular transcripts: the platform establishes an auditable, enterprise-grade university ecosystem.

All six measurable project objectives were completely satisfied, evidenced by 316 passing automated tests across 24 suites, 88.4% line coverage, zero open S1/S2 defects, and a highly responsive single-page application deployed to production on Vercel (`cc54.vercel.app`).

Future Work and Roadmap

1. **Native Mobile App Companion:** Develop Flutter / React Native cross-platform mobile binaries utilizing on-device camera hardware and biometric Secure Enclave for instant QR attendance scanning.
2. **Payment Gateway Integration:** Incorporate SSLCommerz / bKash merchant APIs to support paid professional conferences and merchandise ticketing with automated financial reconciliation.
3. **RFID / NFC Campus Card Integration:** Interface venue scanners with university smart cards (MIFARE / NFC) to allow dual verification (card tap + dynamic QR).
4. **Institutional LMS Integration:** Implement LTI (Learning Tools Interoperability) standards to synchronize club co-curricular attainments directly with institutional student academic databases.

5.2 Reflection

The execution of CampusConnect reinforced the immense value of a **design-first engineering methodology**. Spending the first six weeks rigorously analyzing domain models, specifying formal requirements under IEEE Std 830-1998, and modeling database transactions prevented costly architectural rework during subsequent implementation phases.

A key learning arose during our interaction with AI assistant tools: while AI tools accelerated boilerplate generation, they initially suggested optimistic concurrency patterns for seat booking that failed under simultaneous burst testing. By critically auditing tool suggestions against database concurrency principles, the team rejected optimistic locking in favor of pessimistic transaction serialization, successfully eliminating race conditions.

Working as a collaborative two-person team fostered mutual accountability, deep code review hygiene, and professional pride in delivering a reliable, ethical, and transformative software platform for the UITS academic community.

References

[1] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998. [2] ISO/IEC/IEEE, "Systems and software engineering - Life cycle processes - Requirements engineering," ISO/IEC/IEEE 29148:2018, 2018. [3] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2020. [4] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2015. [5] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002. [6] D. K. Soper, "Greengard's Thesis and Concurrency Control in Transactional Database Systems," *ACM Computing Surveys*, vol. 51, no. 4, pp. 78-95, 2019. [7] D. M. Kroll and R. Rivest, "Hash-Based Dynamic Message Authentication Protocols," *Journal of Cryptology*, vol. 34, no. 2, pp. 112-129, 2021. [8] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA) in JSON Object Signing and Encryption (JOSE)," IETF RFC 8037, Jan. 2017. [9] Board of Accreditation for Engineering and Technical Education (BAETE), "Manual for Accrediting Undergraduate Engineering Programmes," Institution of Engineers, Bangladesh (IEB), 2023. [10] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 2023. [11] GitHub, "GitHub Actions documentation," GitHub Docs. [Online]. Available: `https://docs.github.com/actions` (accessed Oct. 31, 2026). [12] Vitest, "Vitest: Next Generation Testing Framework," Vitest Dev. [Online]. Available: `https://vitest.dev` (accessed Oct. 31, 2026).

Appendix A: Tool Usage Log

In accordance with course academic integrity guidelines, Table A.1 logs every significant utilization of assistant AI tools across the project lifecycle.

Table A.1. Assistant Tool Usage Log

Wk	Task / Artifact	Tool Used	Own Design First (What We Did)	Tool's Role (Refine / Explore / Verify)	How Output Was Verified	Decision (Accepted / Modified / Rejected)
W3	SRS Data Model Specification	Claude 3.5 Sonnet	Drafted initial 14 relational entities and primary/foreign key mappings.	Explored normalization boundary cases (3NF) and index recommendations.	Verified against relational schema rules and foreign key integrity.	Modified: Added explicit unique constraints on `(club_id, user_id)`.
W5	DDL Schema Migration Scripts	GitHub Copilot	Authored SQL table creation templates and data types.	Auto-completed repetitive column constraints and foreign key syntax.	Executed SQL script in SQLite and PostgreSQL; checked schema tables.	Accepted: Cleaned syntax and ensured parameter placeholders `$1, $2`.
W8	Concurrency Seat Booking Engine	Claude 3.5 Sonnet	Architected transactional booking sequence with capacity check.	Suggested optimistic locking vs pessimistic serialization.	Simulated 10 concurrent requests using Vitest multi-worker execution.	Rejected optimistic lock; Accepted pessimistic serialization.
W9	Dynamic QR Attendance Protocol	Claude 3.5 Sonnet	Designed 6-digit rotating OTP concept with 20-second validity.	Recommended HMAC-SHA256 sliding window with `slot:hmac` payload.	Tested token generation, clock skew (+/- 20s), and replay rejection.	Accepted: Integrated 20s HMAC-SHA256 sliding window token.
W10	PDF Certificate & Transcript Generator	Claude 3.5 Sonnet	Created PDFKit layout and QR code positioning coordinates.	Refined stream piping to calculate SHA-256 and Ed25519 digital digests.	Verified hash matches downloaded PDF file digest using `sha256sum`.	Accepted: Embedded verification QR and cryptographic signature calculation.
W11	Automated Vitest Suites	Claude 3.5 Sonnet	Wrote test specifications for authentication and event lifecycle.	Drafted mock request wrappers for Supertest Express endpoint testing.	Executed `npm test`; verified all 316 test assertions passed green.	Modified: Added test rate-limiter bypass to prevent false 429s.

Wk	Task / Artifact	Tool Used	Own Design First (What We Did)	Tool's Role (Refine / Explore / Verify)	How Output Was Verified	Decision (Accepted / Modified / Rejected)
W13	Final Project Report Sweep	Grammarly & Linter	Authored complete 24-page report following course template.	Swept text for spelling, passive voice, and typography consistency.	Checked for zero long hyphens (em-dashes, en-dashes) and validated IEEE table layouts.	Accepted: Standardized on ASCII hyphens, colons, and pipes.

Appendix B: Team Contribution and Peer Evaluation

Repository: `https://github.com/sakib-cse54/cammpus-connect` (Local: `/home/tr/Desktop/project/sakib/codes`) Deployed Product: `https://cc54.vercel.app` (Production) \| `http://localhost:5000` (Unified)

Table B.1. Team Contribution Summary

Member (Name, ID)	Modules / Tasks Owned	Commits / Pull Requests	Documentation, Testing and Review Role
Md. Shakhawat Hossain (ID: 0432320005101009)	Core system architecture, relational database DDL, ACID concurrency seat allocation, waitlist promotion state machine, Ed25519 transcript signing engine, BAETE PO attainment aggregator, JWT authentication & RBAC middleware, Admin Console & Advisor Governance portals, Vercel deployment.	56 commits 19 pull requests	Authored SDD v1.0 and Software Test Plan; engineered Vitest concurrency race-condition suites; reviewed all frontend PRs.
Fahim Hossain (ID: 0432320005101007)	Anti-proxy dynamic rotating QR code engine, mobile deep-link check-in, 6-digit short code fallback, PDFKit vector certificate compiler, React 19 single-page application, Student Portal, Executive Desk, Public Certificate Verification portal, Co-Curricular Transcript Verification desk.	53 commits 17 pull requests	Authored IEEE SRS v2.0 and Deployment Runbook; implemented unit tests for QR, check-in, and certificates; reviewed backend API PRs.

Table B.2. Agreed Contribution Split Certification

Member (Name, ID)	Agreed Contribution (%)	Signature	Date
Md. Shakhawat Hossain (ID: 0432320005101009)	50%	*Md. Shakhawat Hossain*	31 October 2026
Fahim Hossain (ID: 0432320005101007)	50%	*Fahim Hossain*	31 October 2026

The two percentages add up to exactly 100%. Both engineering members contributed equally across architectural planning, frontend/backend implementation, automated quality assurance, and formal documentation.

Table B.3. Peer Evaluation Matrix (4 = Excellent, 1 = Needs Work)

Evaluation Criterion	Member 1 (Shakhawat) rates Member 2 (Fahim)	Member 2 (Fahim) rates Member 1 (Shakhawat)
Technical Contribution	4 (Excellent)	4 (Excellent)
Reliability and Meeting Deadlines	4 (Excellent)	4 (Excellent)
Communication and Collaboration	4 (Excellent)	4 (Excellent)
Code Review and Quality of Work	4 (Excellent)	4 (Excellent)
Peer Comments	"Fahim demonstrated outstanding skill in designing the dynamic QR attendance mechanism, mobile check-in flows, and building responsive UI components."	"Shakhawat provided exceptional architectural leadership, ensuring our concurrency booking engine was bulletproof and implementing the EdDSA transcript signer."

Appendix C: Supporting Documents

The supporting documentation artifacts produced during the semester are stored in the project workspace `docs/` folder:

- **Software Requirements Specification (SRS v2.0):** Comprehensive formal specification conforming to IEEE Std 830-1998 (`docs/CampusConnect_SRS_v2.pdf` / `docs/CampusConnect_SRS_v2.docx`).
- **Software Design Document (SDD v1.0):** Detailed modular architecture, database DDL schemas, and interface definitions (`docs/CampusConnect_SDD_v1.pdf`).
- **Software Test Report (STR v1.0):** Complete testing report detailing 316 passing tests, traceability matrix, and coverage metrics (`docs/CampusConnect_Software_Test_Report.pdf`).
- **Deployment Runbook & Operations Guide:** Production setup, Vercel configuration, Docker Compose, and maintenance manual (`docs/CampusConnect_Deployment_Runbook.pdf`).
- **REST API Specification (v3.0):** Complete endpoint contracts, JSON schemas, EdDSA signature format, and error codes (`docs/CampusConnect_API_Specification.pdf`).
- **README / User Guide:** Repository overview, local installation instructions, test execution commands, and product walkthrough (`README.md`).

Evaluation Sheet (Instructor Use Only)

[Leave this page blank. The instructor records marks for each member here; CO attainment is recorded per student.]

Table E.1. Final Report Evaluation

Criterion	Evidence in Report	CO	Marks	Member 1 (Shakhawat)	Member 2 (Fahim)
Problem analysis, SRS and design	Ch. 2; Table 4.1	CO1	15		
Investigation and design rationale	2.1, 2.4	CO5	10		
Implementation and use of modern tools	Ch. 3; App. A	CO2	10		
Testing and quality assurance	4.1-4.2; App. C	CO3	15		
Deployment, CI/CD and project management	4.3-4.4	CO4	15		
Professional and ethical responsibility	4.2; AI Ack.; App. A	CO8	10		
Teamwork and individual contribution	1.3; App. B	CO7	10		
Communication: structure, clarity, figures, references	Whole report; App. C	CO6	15		
Total Marks			**100**		

Instructor: Al Imtiaz, Assistant Professor, Department of CSE **Signature & Date:** _____